

OzZoo Design Pattern Documentation

Observer, Factory, and Core OOP Patterns (excluding

`lab_work_07_school_simulator.py`)

Scope

This document covers the **Observer**, **Factory**, **Encapsulation**, **Inheritance**, **Polymorphism**, and **Abstraction** patterns implemented in the OzZoo codebase only (`ozzoo/` package), excluding `lab_work_07_school_simulator.py` .

Observer Pattern 1: Animal-level Subject/Observer

Pattern used: Observer (classic subject-observer relationship)

Primary locations:

- `ozzoo/models/animal.py`
- `ozzoo/interfaces/base.py` (observer/observable contracts)

What is implemented

`Animal` acts as the **Subject** with:

- `attach(observer)`
- `detach(observer)`
- `_notify_observers(event)`

The model emits lifecycle/state events such as:

- `"starving"`
- `"stressed"`
- `"sick"`
- `"recovered"`
- `"death"`
- `"escape"`
- `"fed"`

Why this pattern was chosen

Animal state transitions happen frequently and should not hard-code UI/game-manager behavior inside the `Animal` model. Observer allows `Animal` to remain focused on domain rules while external systems react independently.

How it improves the design

1. **Low coupling:** `Animal` does not need to know who listens.
2. **High extensibility:** New observers (UI alerts, analytics, achievements) can be added without editing `Animal` logic.
3. **Better separation of concerns:** Core simulation behavior is isolated from side effects.

Example in this codebase

When health crosses a critical threshold or sickness/recovery changes, `Animal` emits observer events rather than directly triggering UI/game actions.

Observer Pattern 2: Central Event Bus (`EventManager`)

Pattern used: Observer with publish/subscribe event bus

Primary locations:

- `ozzoo/patterns/observer.py`
- `ozzoo/models/zoo.py` (event emission usage)

What is implemented

`EventManager` is a centralized publisher that supports:

- `subscribe(event_type, observer)`
- `unsubscribe(event_type, observer)`
- `emit(...)` / `notify(...)`
- event history storage (`ZooEvent`)

Supporting observer implementations:

- `LoggingObserver`
- `CallbackObserver`

Event type constants are centralized in `EventTypes` (death, birth, sick, recovered, day_start, day_end, purchase, donated, etc.).

Why this pattern was chosen

Zoo-wide events (day cycle, purchases, donations, sickness, death, random events) are cross-cutting concerns. A central pub/sub channel avoids tight coupling between every producer and consumer.

How it improves the design

1. **Scalable communication:** Multiple systems can react to one event without direct dependencies.
2. **Event filtering:** Subscribers can listen to specific event types or wildcard subscriptions.
3. **Traceability:** Event history supports debugging and reporting.

Example in this codebase

`Zoo.advance_day()` emits events like `DAY_START` , `ANIMAL_DEATH` , `ANIMAL_SICK` , `VISITOR_DONATED` , and `DAY_END` , letting observers react independently.

Factory Pattern 1: `AnimalFactory`

Pattern used: Factory Method / Simple Factory

Primary location: `ozzoo/patterns/factory.py`

What is implemented

`AnimalFactory` centralizes creation of concrete animal subclasses from a normalized type string:

- `create_animal(animal_type, **kwargs)`
- `_animal_types` map (type key -> class)
- metadata map (`_animal_info`) used by UI/shop
- extensibility hook (`register_type`)

Also exposed via convenience function:

- `create_animal(...)`

Why this pattern was chosen

Animal creation is required from multiple contexts (shop purchases, loading saves, breeding, starter zoo generation). A factory avoids repeated class-selection logic and inconsistent constructor handling.

How it improves the design

1. **Single creation point:** Constructor logic and normalization rules are centralized.
2. **Consistency:** All callers create animals in the same validated way.
3. **Open for extension:** New species can be added by registration/mapping without changing every caller.

Example in this codebase

`Zoo.purchase_animal(...)` and save-load reconstruction in `zoo.py` call `AnimalFactory.create_animal(...)` instead of directly instantiating species classes.

Factory Pattern 2: `create_enclosure(...)` Factory Function

Pattern used: Factory Function

Primary location: `ozzoo/models/enclosure.py`

What is implemented

A dedicated factory function creates enclosure subclasses from habitat type:

- `ENCLOSURE_TYPES` map (habitat key -> class)
- `create_enclosure(habitat_type, enclosure_id, **kwargs)`

Why this pattern was chosen

Enclosures are polymorphic and selected dynamically by habitat requirements (build flow, save-load flow, starter setup). A factory avoids scattered conditional creation logic.

How it improves the design

1. **Centralized type resolution:** One authoritative place maps habitat to subclass.
2. **Safer construction:** Validates unknown habitat types in one location.
3. **Cleaner callers:** Zoo/game setup code stays focused on gameplay flow, not subclass selection.

Example in this codebase

`Zoo.build_enclosure(...)` and deserialization (`Enclosure.from_dict(...)`) both rely on `create_enclosure(...)` .

Encapsulation Pattern: Protected Internal State via Properties

Pattern used: Encapsulation (information hiding + controlled mutation)

Primary locations:

- `ozzoo/models/animal.py`
- `ozzoo/models/enclosure.py`

- `ozzoo/models/zoo.py`

What is implemented

Core domain classes store state in private/protected-style fields (`_health` , `_hunger` , `_animals` , `_budget` , etc.) and expose controlled access through properties and methods.

Examples:

- `Animal.health` , `Animal.hunger` , `Animal.happiness` setters clamp values to valid ranges and trigger side effects/events.
- `Enclosure.animals` returns a **copy** of the internal list, preventing accidental external mutation.
- `Zoo.animals` , `Zoo.enclosures` , `Zoo.visitors` , resource stock properties return copies rather than raw internals.

Why this pattern was chosen

Zoo simulation state is shared by many systems (UI, missions, events, feeding, reviews). Direct unrestricted writes would make state corruption and rule bypasses easy.

How it improves the design

1. **Invariant protection:** Health/hunger/happiness and cleanliness remain bounded and valid.
2. **Safer API boundaries:** Callers cannot mutate critical collections unintentionally.
3. **Centralized business rules:** Side effects (death/sick/stressed notifications) are enforced in one place.

Example in this codebase

`Animal.health` enforces 0-100 bounds and triggers `"death"` / `"sick"` transitions when thresholds are crossed, rather than allowing arbitrary external assignment.

Inheritance Pattern 1: Animal Hierarchy

Pattern used: Inheritance (multi-level domain hierarchy)

Primary locations:

- `ozzoo/models/animal.py`
- `ozzoo/models/animals/mammals.py`
- `ozzoo/models/animals/birds.py`
- `ozzoo/models/animals/reptiles.py`

What is implemented

OzZoo uses a layered class hierarchy:

- `Animal` (abstract base class)
- Intermediate categories: `Mammal` , `Bird` , `Reptile`
- Concrete species: `Koala` , `Kangaroo` , `Wombat` , `Platypus` , `Kookaburra` , `Emu` , `SaltwaterCroc` , etc.

Why this pattern was chosen

All species share core behaviour (health/hunger/happiness, feeding, daily updates) but also need category/species-specific fields and actions.

How it improves the design

1. **Code reuse:** Shared behaviour lives in `Animal` /intermediate classes instead of being duplicated in each species.
2. **Clear taxonomy:** The class model mirrors real domain relationships.
3. **Extensibility:** New species can be added by extending the appropriate base class.

Example in this codebase

`Koala` , `Emu` , and `SaltwaterCroc` inherit common animal lifecycle logic but override species-specific methods like `make_sound()` , `get_diet()` , and `get_habitat_type()` .

Inheritance Pattern 2: Enclosure and Exception Hierarchies

Pattern used: Inheritance for specialization

Primary locations:

- `ozzoo/models/enclosure.py`
- `ozzoo/exceptions/zoo_exceptions.py`

What is implemented

- `Enclosure` is an abstract parent class with concrete subclasses (`EucalyptusGrove` , `OutbackSavanna` , `Billabong` , `RainforestAviary` , `ReptileHouse`).
- Exception types derive from `ZooError` into focused branches (`AnimalError` , `ResourceError` , `EnclosureError`) and then concrete errors.

Why this pattern was chosen

Both habitat logic and error handling need shared behavior plus targeted specialization.

How it improves the design

1. **Consistent behavior:** Shared enclosure/error behavior is centralized in base classes.
2. **Specific semantics:** Subclasses provide focused actions/messages without duplicating base structure.
3. **Cleaner handling:** Callers can catch broad or specific exception categories as needed.

Example in this codebase

`InvalidHabitatError` and `HabitatCapacityExceededError` both inherit from `EnclosureError` , enabling enclosure-specific error handling with meaningful specialized messages.

Polymorphism Pattern 1: Uniform Animal Handling

Pattern used: Polymorphism (treating different species through a common interface)

Primary locations:

- `ozzoo/models/zoo.py`
- `ozzoo/models/animal.py` and concrete species modules

What is implemented

Zoo logic works with `Animal` references and calls shared methods/properties (`daily_update()` , `feed()` , `get_diet()` , `is_alive`) without knowing concrete species types.

Why this pattern was chosen

Day-cycle and feeding systems must work for many species without long `if/elif` chains per class.

How it improves the design

1. **Generic game logic:** Core systems operate on common animal contracts.
2. **No species branching explosion:** Adding species does not require rewriting zoo loops.
3. **Runtime flexibility:** Correct overridden behavior is selected automatically per concrete class.

Example in this codebase

`Zoo.advance_day()` iterates through `self._animals.values()` and calls `animal.daily_update()` for all animals uniformly, regardless of species.

Polymorphism Pattern 2: Uniform Enclosure Handling

Pattern used: Polymorphism across habitat subclasses

Primary locations:

- `ozzoo/models/zoo.py`
- `ozzoo/models/enclosure.py`

What is implemented

Zoo code treats enclosures through the base `Enclosure` API (`add_animal()`, `daily_update()`, `clean()`, `get_habitat_type()`), while each subtype provides distinct implementations.

Why this pattern was chosen

Different habitats require different cleaning and compatibility logic but must still plug into one daily simulation pipeline.

How it improves the design

1. **Consistent orchestration:** One loop can update all enclosures.
2. **Specialized behavior:** Each enclosure class encapsulates habitat-specific rules.
3. **Easier feature growth:** New habitat subclasses integrate without changing core zoo flow.

Example in this codebase

`Zoo.advance_day()` calls `enclosure.daily_update()` in one loop, and each enclosure subtype applies its own internals while exposing the same interface.

Abstraction Pattern 1: Abstract Domain Base Classes

Pattern used: Abstraction via abstract base classes (ABCs)

Primary locations:

- `ozzoo/models/animal.py`
- `ozzoo/models/enclosure.py`

What is implemented

`Animal` and `Enclosure` define abstract methods that capture essential contracts:

- `Animal` : `make_sound()`, `get_diet()`, `get_habitat_type()`, `get_animal_type()`

- Enclosure : `clean()` , `get_habitat_type()` , `get_compatible_species()`

Why this pattern was chosen

Game systems need guaranteed behavior from all animal/enclosure types while hiding concrete implementation details.

How it improves the design

1. **Clear contracts:** Subclasses must implement required methods before use.
2. **Reduced complexity for callers:** Code depends on what objects can do, not how they do it.
3. **Safer extension:** Incomplete subclasses are caught during implementation.

Example in this codebase

Concrete species cannot be added without implementing `get_habitat_type()` , ensuring compatibility checks in enclosure placement always have required data.

Abstraction Pattern 2: Interface Contracts in `interfaces/base.py`

Pattern used: Interface-style abstraction

Primary location: `ozzoo/interfaces/base.py`

What is implemented

Reusable contracts define expected behavior for core concerns:

- `ICleanable` (`clean` , `get_cleanliness`)
- `IFeedable` (`feed` , `get_diet` , `is_hungry`)
- `IObservable` / `IObserver` (observer protocol methods)

Why this pattern was chosen

These contracts separate **what behavior is required** from any one concrete implementation, enabling consistent integration across modules.

How it improves the design

1. **Architecture clarity:** Shared behavior expectations are explicit and centralized.
2. **Loose coupling:** Implementations can vary without changing contract consumers.
3. **Improved maintainability:** Future refactors can preserve contracts while changing internals.

Example in this codebase

Observer-capable components can be integrated consistently because the observable/observer interaction is defined at the interface level.

Why these patterns together are effective in OzZoo

These patterns work in layers:

- **Abstraction + Inheritance** define the domain contracts and class hierarchies.
- **Encapsulation** protects simulation invariants and state integrity.
- **Polymorphism** lets the day-cycle/gameplay logic handle diverse objects uniformly.
- **Factory** standardizes object creation.

- **Observer** standardizes event propagation.

Together, they reduce coupling, improve extensibility, and keep gameplay logic scalable as the zoo grows.